Data Mining Assignment I

Divine Mahoro
ID: 101199
Group 2

Chapter 2: Statistical Learning Question 1 (Textbook Exercise 2.2)

Explain whether each scenario is a classification or regression problem, and indicate whether we are most interested in inference or prediction. Finally, provide n and p.

Answer:

1. **Scenario (a): CEO Salary Analysis**

   - **Problem Type: Regression**. The target variable (CEO Salary) is a continuous numerical measurement.
   - **Primary Interest: Inference**. The objective is to understand how different factors affect the CEO salary.
   - **Dimensions:** n = 500 firms, p = 3 predictors (profit, number of employees, and industry type).

2. **Scenario (b): Product Launch Evaluation**

   - **Problem Type: Classification**. The output categorical, consisting of two qualitative states: Success or Failure.
   - **Primary Interest: Prediction**. The goal is to build a model capable of projecting the future outcome of a product being launched.
   - **Dimensions:** n = 20 historical products, p = 13 predictors (price, marketing budget, competition price, and 10 additional documented variables).

3. **Scenario (c): Exchange Rate Forecasting**

- **Problem Type: Regression**. The target output (percentage change in USD/Euro rate) is a continuous numeric value.
- **Primary Interest: Prediction**. The focus is to forecast currency market fluctuations from past weekly changes.
- **Dimensions:** n = 52 data points (representing the 52 weekly intervals within the full calendar year of 2012), p = 3 predictors (percentage changes of the US, British, and German stock markets).

Question 2 (Textbook Exercise 2.9)

This exercise involves the Auto data set studied in the lab. Make sure that the missing values have been removed from the data

(a) Variable Classifications

Based on the structure of the Auto new dataframe, the predictors are classified into the following statistical categories:

- **Quantitative Predictors:** mpg, cylinders, displacement, horsepower, weight, acceleration, and year. These variables consist of continuous numerical values representing physical measurements or time.

- **Qualitative Predictors:** name (categorical text strings for car model descriptions) and origin (integers acting as qualitative indicator flags for manufacturing regions).

(b) Range of Quantitative Predictors

By applying the NumPy statistical functions np.min() and np.max() to each quantitative variable in the cleaned Auto new dataset, the exact observational ranges are calculated as follows:

- mpg**:** Min = 9.0, Max = 46.6, Range = [9.0, 46.6]
- cylinders**:** Min = 3, Max = 8, Range = [3, 8]
- displacement**:** Min = 68.0, Max = 455.0, Range = [68.0, 455.0]
- horsepower**:** Min = 46.0, Max = 230.0, Range = [46.0, 230.0]
- weight**:** Min = 1613.0, Max = 5140.0, Range = [1613.0, 5140.0]
- acceleration**:** Min = 8.0, Max = 24.8, Range = [8.0, 24.8]
- year**:** Min = 70, Max = 82, Range = [70, 82]

(c) Mean and Standard Deviation of Quantitative Predictors

Using the statistical calculation modules np.mean() and np.std() within NumPy, the sample means and standard deviations across all quantitative metrics are evaluated as follows:

- mpg**:** Mean = 23.45, Std Dev = 7.81
- cylinders**:** Mean = 5.47, Std Dev = 1.71
- displacement**:** Mean = 194.41, Std Dev = 104.64
- horsepower**:** Mean = 104.47, Std Dev = 38.49
- weight**:** Mean = 2977.58, Std Dev = 849.40
- acceleration**:** Mean = 15.54, Std Dev = 2.76
- year**:** Mean = 75.98, Std Dev = 3.68

(d) Summary Statistics (Observations 10–85 Removed)

After dropping the 10th through 85th row entries (indices 9 to 84 in
Python's 0-indexed configuration), the structural descriptive metrics shift to the following parameters for the remaining *n* = 316 vehicle observations:

- mpg**:** Range = [11.0, 46.6], Mean = 24.40, Std Dev = 7.87
- cylinders**:** Range =, Mean = 5.37, Std Dev = 1.65
- displacement**:** Range = [68.0, 455.0], Mean = 187.24, Std Dev = 99.68
- horsepower**:** Range = [46.0, 230.0], Mean = 100.72, Std Dev = 35.71
- weight**:** Range = [1649.0, 4997.0], Mean = 2935.97, Std Dev = 811.30

- acceleration**:** Range = [8.5, 24.8], Mean = 15.73, Std Dev = 2.69
- year**:** Range =, Mean = 77.15, Std Dev = 3.11

(e) Graphical Analysis and Predictor Relationships

By implementing graphical scatterplot matrices, correlation heatmaps, and trend visualizations across the complete Auto new dataset, several strong linear and non-linear patterns emerge between the variables:

- **High Positive Multicollinearity:** Engine displacement, engine horsepower, and vehicle weight are heavily tied together in strong positive linear trends. This is mathematically validated by the correlation matrix, which reveals exceptionally high correlation parameters between these design elements, ranging from 0.84 up to 0.95. This confirms that vehicles configured with larger engine volumes naturally scale upwards in mass and absolute power output.
- **Negative Inverted Trajectories:** Plotting acceleration against metrics like vehicle weight or engine horsepower shows a distinct downward slope ($r = -0.42$ and $r = -0.69$, respectively). This reflects a mechanical tradeoff where heavier cars and smaller engines yield slower overall acceleration times.
- **Categorical Layout Shifts:** Boxplots constructed across discrete attributes like cylinders show that vehicles

grouped into larger engine classes (6 and 8 cylinders) display compressed variation fields localized around much higher baseline weights and displacement measurements.

(f) Evaluation of Predictors for Fuel Economy (mpg**)**

The graphical distributions strongly demonstrate that multiple quantitative features within the dataset serve as excellent, highly useful predictors for forecasting a vehicle's gas mileage (mpg):

- weight**,** displacement**, and** horsepower**:** These variables display intense, clear negative non-linear relationships when plotted against mpg, backed by high negative correlation scores ($r = -0.83$, $r = -0.81$, and $r = -0.78$). The steep downward-sloping curves confirm that as a vehicle scales up in structural mass, volume, or power, its overall fuel efficiency drops precipitously.

- year**:** Scatterplots and boxplots mapping mpg across time show a steady, clear upward trend line ($r = 0.58$). This acts as clean data evidence documenting historical mechanical improvements over the years—newer vehicle models achieve significantly higher mileage than older ones under the same weight thresholds.

- cylinders**:** Clear gaps in categorical groupings confirm that higher cylinder counts correspond with lower gas efficiency marks, isolating 8-cylinder vehicles to the lower bounds of the mpg axis.

In conclusion, a multi-variable regression approach incorporating vehicle weight, historical model year, and performance metrics like horsepower will deliver an exceptionally robust prediction machine for gas mileage estimation.

Chapter 3: Linear Regression Question 1 (Textbook Exercise 3.2)

Carefully explain the differences between the KNN classifier and KNN regression methods.

They both predict the outcome of an input based on the closest neighbor, and then they differ on how they conclude the prediction:

- **KNN Classifier:** Counts how many points are near to the unknown point and the value they find to be many is the outcome of the prediction. This type is mostly used when we are working with categorical values.
- **KNN Regressor:** Takes the average number of points near the unknown and that computed average becomes the value of the outcome for that point. This type is mostly used when working with continuous values.

Data Transformation Note

If you have continuous data and want to use a classifier (and vice-versa), it is not completely impossible. However, you would need to transform the data first by converting the

exact numbers into category brackets or converting categories into binary scores.

Question 2 (Textbook Exercise 3.7)

It is claimed in the text that in the case of simple linear regression of Y onto X, the R^2 statistic is equal to the square of the correlation between X and Y . Prove that this is the case. For simplicity, you may assume that $\bar{x} = \bar{y} = 0$.

Proof

Step 1: Core Definitions with Center Data ($\bar{x} = 0, \bar{y} = 0$)
When the sample means are zero, the squared sample correlation coefficient (r^2) is defined as:

$$r^2 = \frac{(\sum x_i y_i)^2}{\sum x_i^2 \sum y_i^2}$$

The R^2 statistic is defined using the Total Sum of Squares (TSS) and Residual Sum of Squares (RSS):

$$R^2 = 1 - \frac{RSS}{TSS} = \frac{TSS - RSS}{TSS}$$

Since $\bar{y} = 0$, the TSS simplifies to:

$$TSS = \sum y_i^2$$

Step 2: Simple Linear Regression Framework

The simple linear regression line is $\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_i$. Given $\bar{x} = \bar{y} = 0$, the intercept becomes $\hat{\beta}_0 = 0$, leaving the prediction equation as:

$$\hat{y}i = \hat{\beta}1xi$$

Where the optimal least-squares slope estimate ($\hat{\beta}_1$) is defined as:

$$\hat{\beta}_1 = \frac{\sum x_i y_i}{\sum x_i^2} \implies \hat{\beta}_1 \sum x_i^2 = \sum x_i y_i$$

Step 3: Expanding and Simplifying RSS We expand the RSS using our prediction line:

$$\text{RSS} = {}^{X}(y_i - \hat{y}_i)^2$$

$$= X(yi - \hat{\beta}1xi)2$$

$$= Xyi2 - 2\hat{\beta}1Xxiyi + \hat{\beta}12Xx2i$$

By splitting the squared term into $\hat{\beta}_1 \left(\hat{\beta}_1 \sum x_i^2\right)$ ${}^{P}x_i y_i$, one $\hat{\beta}_1$ factor is absorbed:

and substituting $\hat{\beta}_1{}^{P}x^2_i =$

$$\text{RSS} = \sum y_i^2 - 2\hat{\beta}_1 \sum x_i y_i + \hat{\beta}_1 \left(\sum x_i y_i\right)$$

$$= Xyi2 - \hat{\beta}1Xxiyi$$

Step 4: Deriving the Final R^2 Identity

Now, we substitute our simplified expression for RSS and TSS back into the R^2 equation:

2 TSS − RSS *R* =

TSS

$$= \frac{\sum y_i^2 - \left(\sum y_i^2 - \hat{\beta}_1 \sum x_i y_i\right)}{\sum y_i^2}$$

$$= \frac{\hat{\beta}_1 \sum x_i y_i}{\sum y_i^2}$$

Finally, substituting the full definition of $\hat{\beta}_1 = \frac{\sum x_i y_i}{\sum x_i^2}$ yields:

$$R^2 = \frac{\left(\frac{\sum x_i y_i}{\sum x_i^2}\right) \sum x_i y_i}{\sum y_i^2} = \frac{\left(\sum x_i y_i\right)^2}{\sum x_i^2 \sum y_i^2}$$

Comparing our calculated R^2 to the core definition of r^2 from Step 1, we establish that:

$$R2 = r2$$

This completes the proof. ■

Question 3 (Textbook Exercise 3.12)

This problem involves simple linear regression without an intercept.

(a) Condition for Identical Coefficients

The coefficient estimate $\hat{\beta}_{YX}$ for the regression of Y onto X without an intercept is given by:

$$\hat{\beta}_{YX} = \frac{\sum_{i=1}^{n} x_i y_i}{\sum_{i=1}^{n} x_i^2}$$

Similarly, flipping the roles to regress X onto Y yields the coefficient estimate $\hat{\beta}_{XY}$:

$$\hat{\beta}_{XY} = \frac{\sum_{i=1}^{n} x_i y_i}{\sum_{i=1}^{n} y_i^2}$$

For these two coefficient estimates to be exactly equal ($\hat{\beta}_{YX} = \hat{\beta}_{XY}$), their mathematical denominators must be equal. Therefore, the required circumstance is that the sum of squares of X must equal the sum of squares of Y:

$$\sum_{i=1}^{n} x_i^2 = \sum_{i=1}^{n} y_i^2$$

(b) Python Example: Different Coefficients

Below is the Python code generating $n = 100$ observations where $\sum x_i^2 \neq \sum y_i^2$, causing the two slope parameters to differ:

```
import numpy as np import
statsmodels.api as sm
```

```
np.random.seed(42) x =
np.random.normal(size=100) y = 2 * x +
np.random.normal(size=100)

beta_yx = sm.OLS(y, x).fit().params[0]
beta_xy = sm.OLS(x, y).fit().params[0]

print(f"Beta_YX (Y onto X): {beta_yx:.4f}") # Output: 1.8558
print(f"Beta_XY (X onto Y): {beta_xy:.4f}") # Output: 0.4113
```

(c) Python Example: Identical Coefficients

To guarantee that $^{P}x^{2}_{i} = {}^{P}y_{i}^{2}$ perfectly, we can create *y* by randomly shuffling the exact elements of *x*. This ensures they share identical sums of squares while maintaining a valid data relationship:

```
import numpy as np import
statsmodels.api as sm

x = np.random.normal(size=100) y =
np.random.permutation(x) # Forces sum(x^2) ==
sum(y^2)

beta_yx = sm.OLS(y, x).fit().params[0]
beta_xy = sm.OLS(x, y).fit().params[0]

print(f"Beta_YX (Y onto X): {beta_yx:.4f}") # Output: -0.0433
print(f"Beta_XY (X onto Y): {beta_xy:.4f}") # Output: -0.0433
```